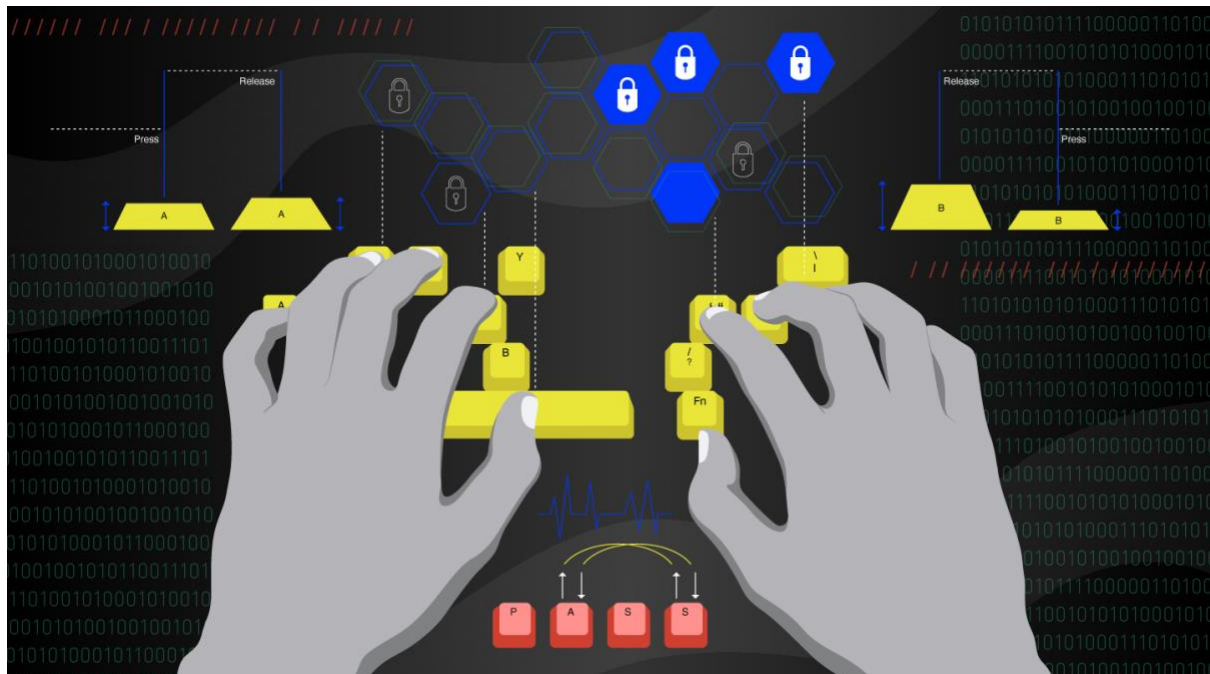




Keystroke-Based Essay Quality Prediction

1. Introduction to the Problem

This report presents our solution to the task of predicting essay quality directly from writers' keystroke logs, without access to the final essay text. The core challenge is deceptively simple to state: given only a raw sequence of keyboard events recorded while a writer drafts an essay, can we reliably estimate how good that essay is? The dataset provides keystroke-level logs capturing every typing action, deletion, pause, cursor movement and paste event during drafting, with all alphanumerical characters anonymised as "q" to prevent direct semantic analysis. Essay scores on a 0-6 scale serve as the prediction target, and submissions are evaluated using Root Mean Squared Error (RMSE). This problem is fundamentally harder than conventional automated essay scoring: because the final text is unavailable, standard natural language processing approaches and pretrained language models cannot be applied directly. Instead, quality signals must be recovered entirely from the temporal and behavioural trace of how the essay was written, including typing speed, pause patterns, revision behaviour and cursor movement. To tackle this, we adopted a dual-modality approach, reconstructing the anonymised final essay from the keystroke log to extract lexical features, while simultaneously mining the raw log for process-level behavioural signals, before combining both modalities into a two-stage stacking ensemble.

2. Exploratory Data Analysis (EDA)

2.1 Score Distribution Analysis

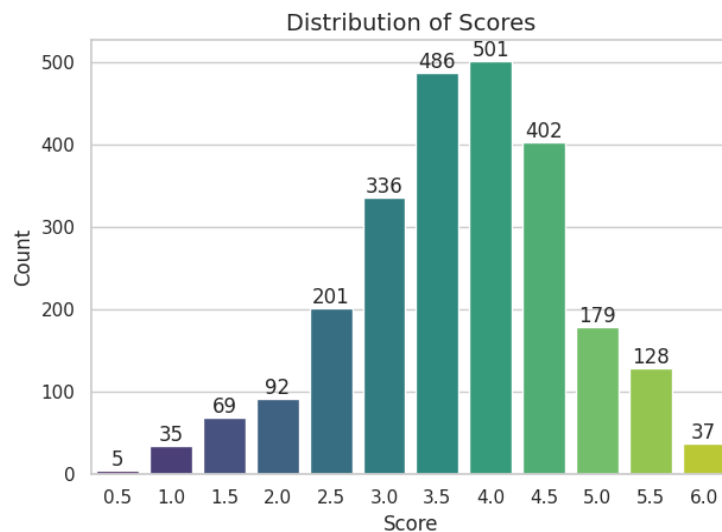


Figure 1: Distribution of training set scores

The distribution of scores is slightly left-skewed, with a median of 4.0 and extreme scores at both tails being underrepresented (1.0-2.0 and 5.0-6.0). Hence, stratified K-fold cross-validation (CV) is used in our training to maintain the original score distribution and ensure extreme scores are sufficiently represented in each fold (Section 6).

2.2 Distribution and Correlation of Features Against Score

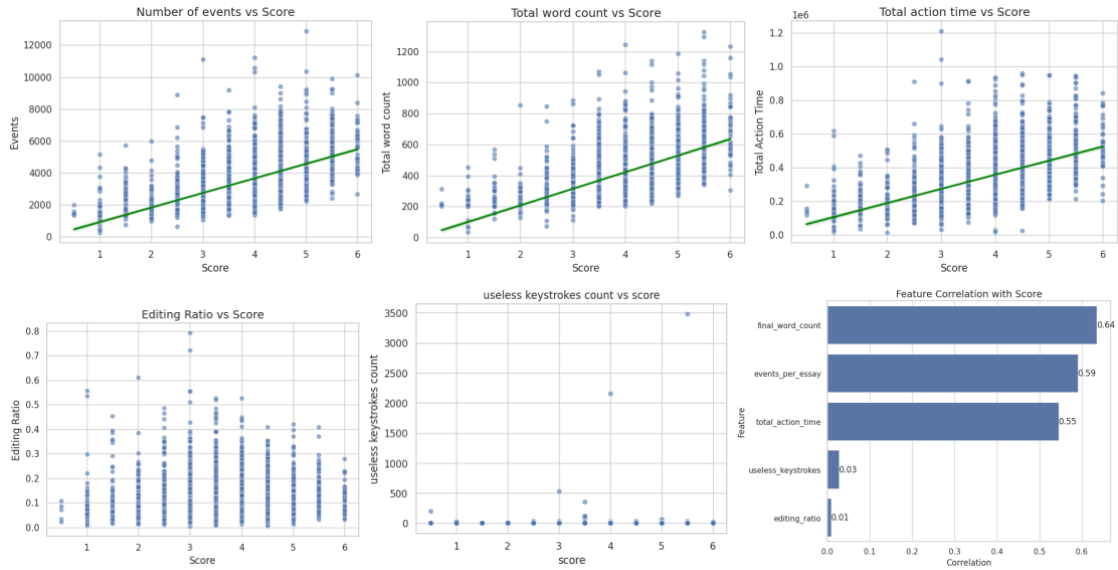


Figure 2: Correlation of features against score

There exists a positive correlation between essay score and event count ($r=0.59$), total word count ($r=0.64$) and total action time ($r=0.55$). Conversely, custom editing ratios involving *Input*, *Remove/Cut*, *Replace* and *Paste* ($r=0.03$) and counts of useless keystrokes involving *Pause* and *AudioVolumeUp* ($r=0.01$) show near-zero correlations with score. This suggests that the mere aggregation of raw events is insufficient and that targeted feature engineering is needed to extract meaningful representations of the writing process.

2.3 Clustering Analysis Using PCA and t-SNE

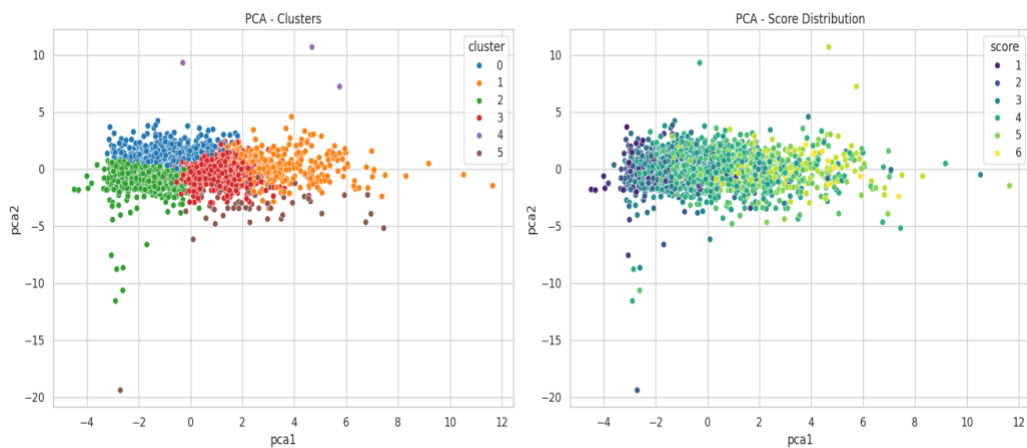


Figure 3: PCA scatter plots visualising clusters and score distribution

K-means clustering ($k=6$) was applied to essay-level aggregated features, such as event count, action time statistics and word count to examine whether distinct writing process patterns exist among writers and if they relate to essay scores. Our PCA shows a slight distinction between

low- and high-scoring writers along the first two principal components, implying that broad behavioural traits may be associated with essay quality.

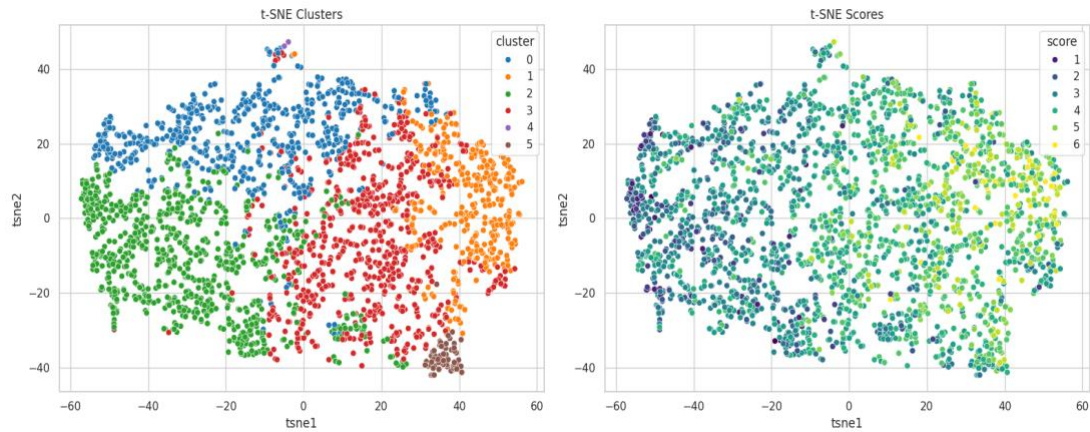


Figure 4: t-SNE scatter plots visualising clusters and score distribution

We also used t-SNE to capture non-linear groupings. However, it showed no clear distinction between score groups. These results suggest that while behavioural patterns carry predictive signals, they may be limited. Thus, more fine-grained feature engineering is required to surface out stronger discriminative features.

2.4 Score Distribution of Clusters

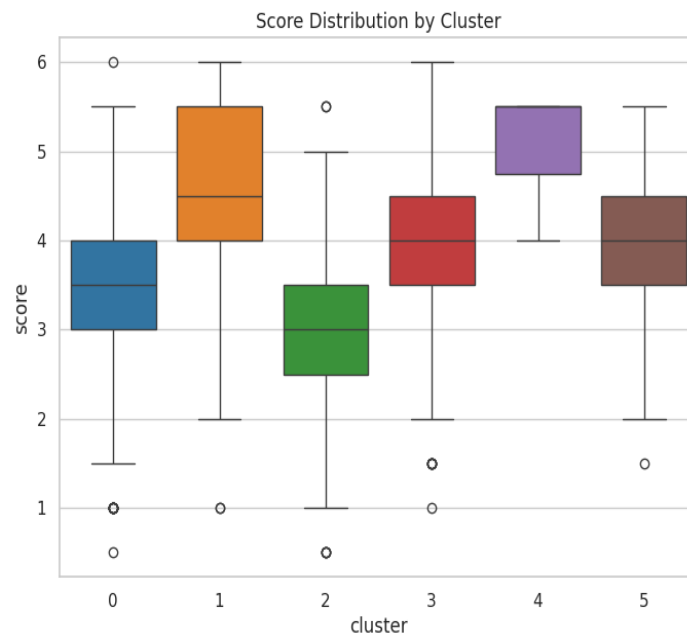


Figure 5: Boxplots of cluster scores

The boxplots show that Clusters 1 and 4 are effective in representing stronger writers and Cluster 2 is effective in capturing weaker writers. Clusters 0, 3 and 5, however, overlap substantially around a median of 3.5-4.0, highlighting that K-means clustering may not reliably

distinguish writing quality. Therefore, while behavioural archetypes exist, they capture writing style more than quality, requiring more robust feature engineering.

3. Key Challenges

3.1 No Direct Access to Essays

Since all alphanumeric characters are masked as “q”, there is no readable text available, rendering methods like natural language processing and pretrained transformer models like BERT inapplicable without essay reconstruction. Therefore, we decided to reconstruct the essays, as it is a prerequisite for lexical feature extraction.

3.2 Missing Ordinal Classification in Regression

Essay scores are continuous and ordered, ranging from 0 to 6, with higher values indicating better performance. However, standard regression models ignore this ordering, producing score predictions that may be numerically close to the ground truth but are incorrectly ranked. To address this, ordinal classification is incorporated into the stacking framework of our model to capture both numerical magnitude and ordinal structure (Section 6).

3.3 Risk of Overfitting Due to High Feature Count

With only 2471 training entries, each represented as a sequence of keystroke events like action time and cursor position, aggregating these sequences into essay-level features may yield a high feature-to-sample ratio, increasing overfitting risk in tree models [1]. Hence, hyperparameter tuning, early stopping and regularised learners are employed (Section 6).

3.4 Timing Noise

The dataset includes key press and release times, like *down_time* and *up_time*, which are not always reflective of actual writing behaviour. Furthermore, long time gaps may occur when writers pause browsers or switch tasks. Coupled with browser throttling and clock drift, these factors can reduce the reliability of data, necessitating us to exclude long downtime values from our training pipeline to prevent these anomalies from skewing our results.

4. Data Cleaning and Pre-Processing

4.1 Log Cleaning & Encoding

To ensure consistent handling of essay identifiers and prevent “unknown-category” errors during inference, writer IDs from both train and test sets are combined and passed to a *LabelEncoder* fitted on the combined set. The preprocessing function *preprocess_log_table* then sorts logs chronologically by *id* and *down_time* for construction of temporal features (eg. *inter-stroke gaps*, *bursts*, *milestone timestamps* in Section 5), reassigns event IDs and removes early and unidentified events to prevent inflation of activity counts. Timestamps are also normalised and text fields are sanitised for consistent encoding. Lastly, sentence and paragraph counts are attached to each row log by examining the reconstructed essay, allowing for pause features to be computed relative to linguistic boundaries instead of global timing.

4.2 Essay Reconstruction

Since final essay texts are unavailable, essay reconstruction is necessary to extract features such as readability, vocabulary richness and TF-IDF (Term Frequency-Inverse Document Frequency). This reconstruction bridges the gap between keystrokes, writing behaviours and the final essay, allowing us to combine content-based and behavioural features for our dual-modality framework for prediction (Section 9.1)

Essay reconstruction was done using *replay_keystrokes*, which processes each writer's keystroke log chronologically while maintaining a mutable string. Each event updates the text as follows:

- (a) **Input:** Inserts text at the cursor position
- (b) **Remove/Cut:** Deletes text at the cursor position
- (c) **Replace:** Atomically swaps original text with revised text at the same cursor position
- (d) **Paste:** Inserts a text block at the cursor position
- (e) **Move:** Relocates contiguous text from its source to target position
- (f) **Nonproduction:** Ignored as these events do not modify essay state (eg. mouse movements, scrolling and idle periods)

After essay reconstruction, *normalise_whitespace* is applied to remove redundant spaces and newlines often caused by repeated keypresses, as they may distort text statistics like sentence length distributions. This yields clean strings suitable for downstream text feature extraction.

5. Feature Engineering

Features are produced from two sources, the reconstructed essay (Sections 5.1-5.5) and the raw keystroke log (Section 5.6), resulting in a matrix of over 300 features for each writer. Several codes were referenced for analysis in our feature engineering and modelling [2] [3].

5.1 Text Surface Features

surface_features computes six metrics from the reconstructed essay: *text_length* (total characters), *word_count* (tokens), *num_newlines* (paragraph breaks), *avg_word_length* (mean characters per word), *word_lexical_diversity* (ratio of unique to total words), and *repetitiveness* (proportion of placeholder character “q”). These are motivated by their empirical strength in Automated Essay Scoring (AES), where surface-level properties at the character, word, sentence and paragraph levels proved to be strong predictors of writing quality [4][5]. In particular, text length and word count are strong indicators of elaboration and nuanced thinking [6], while lexical diversity captures vocabulary sophistication and vocabulary range [7]. In addition, paragraph structure (*num_newlines*) reflects argument organisation, which is a key component of holistic scoring rubrics. *repetitiveness* is unique to this anonymised dataset and acts as a proxy for prose density, where higher values indicate more substantive writing, while lower values may suggest incomplete or list-like responses.

5.2 Word, Sentences & Paragraph Level Statistics

Three tokenisation functions are used to decompose the reconstructed essay into words (*split_to_word*), sentences (*split_to_sentence*) and paragraphs (*split_to_paragraph*). They also filter for tokens containing the anonymising character “q” to retain only meaningful lexical units, avoiding inflation from these placeholders. Such a multi-layer decomposition mirrors the inherent hierarchical structure of argumentative essays, enabling prediction models to capture patterns across different text scales. *agg_stats* then computes ten distributional aggregates (*count, min, max, median, std, sum, first, last, q25, q75*) at each linguistic level. Full distribution metrics were used over a single mean for length because while length is responsible for 24% of additional variance beyond language proficiency [8], features capturing syntactic complexity and structural variation account for an additional 45-49% of variance in argumentative essay scores [9]. Hence, a single aggregate can neglect this variation, as essays of similar length may differ significantly in structure, with essays having more natural variation exhibiting stronger organisation and writing quality [10].

5.3 Grammar & Error Surface Features

In many holistic essay scoring rubrics, correct punctuation use is listed as a key criterion. Stronger writers typically follow standard spacing and formatting and are therefore credited with higher scores, while weaker writers display consistent error patterns like missing spaces or repeated punctuations and are given lower scores [11]. Their incorporation as features also greatly improves AES performance [12]. Therefore, *grammar_surface_error* is implemented, using regular expressions to count common error types such as repeated punctuations, incorrect spacing around commas and periods and total error frequency for each essay. *typo_prob* then estimates the likelihood of typing errors by detecting runs of three or more consecutive consonants in the reconstructed essay (excluding anonymised tokens) indicative of uncorrected typing errors. This feature leverages on keystroke-based reconstruction, capturing artefacts that cannot be found in essays submitted directly.

5.4 TF-IDF Features

Our initial pipeline did not include TF-IDF features, resulting in weak model performance. TF-IDF vectorisers were therefore introduced to encode word importance from reconstructed essays. These include a word n-gram vectoriser (unigrams to trigrams, *sublinear_tf = True, min_df=2*) and a character n-gram vectoriser (2–5 grams, *min_df=2*), as the latter can complement the former by capturing subword patterns like suffixes, prefixes and spelling variations, improving robustness to out-of-vocabulary terms [13]. Since tree models cannot directly process sparse, high-dimensional inputs, SVD is needed to convert them into dense, lower-dimensional embeddings while preserving key variance [14]. Thus, outputs of the vectorisers are concatenated and reduced to 64 components each via *TruncatedSVD*, fitted only on training data. This combination has proved to be effective in related tasks like native language identification, supporting its relevance for writing quality assessment.

A key component of this pipeline is the Ridge regression meta-signal, where a Ridge model is trained on a full word n-gram TF-IDF matrix without SVD compression and uses 10-fold cross-validation to produce out-of-fold (OOF) predictions (*ridge_preds*). This was done as the

model’s L2 regularisation is suitable for high-dimensional sparse data like these, and is capable of capturing direct lexical co-occurrence patterns that non-linear tree models may miss. Predictions (*ridge_preds*) are then fed directly into the Stage 2 meta-learner (Section 6.3), allowing our ensemble model to receive lexical information at two granularities, dense topic-level semantics (SVD) and fine-grained word co-occurrence (Ridge). This dual-granularity lexical representation is a key driver of our competitive performance.

5.5 Keystroke Process Features

These are the most novel components of our pipeline, capturing temporal and behavioural patterns of writing like *how* essays are written independently of *what* was written (Sections 5.1-5.4). In particular, research has shown that writing behaviour, such as typing speed, pause frequency and revision patterns, is strongly associated with writing quality [15].

5.5.1 Timing & Speed Features

The relationship between typing fluency and writing quality is well-established, with measures like total writing time, keystroke count and typing speed being predictive of holistic writing scores [16]. Hence, *compute_typing_rate* is used to estimate writing fluency as *keys_per_second*, defined as the ratio of productive keystrokes (*Input+Remove/Cut*) to total session duration, serving as a macro-level proxy for compositional automaticity. However, a single generic speed estimate may not fully encapsulate writing behaviour, as experienced writers tend to produce text faster with fewer interruptions. *writing_speed_milestones* thus extends this by recording the time taken to reach word counts of 200, 300, 400 and 500, while penalising writers for non-completion. This reveals writing trajectories: a writer slowing down after 300 words may signal difficulty in ideation, while sustained or accelerated speeds may suggest stronger compositional fluency.

5.5.2 Pause Distribution Features

Label	Duration Range	Cognitive Interpretation
Micro	< 100ms	Normal motor rhythm
Short	100-250ms	Lexical access, word retrieval
Medium	250-500ms	Inter-word planning
Moderate	500ms-1s	Local sentence planning
Long	1-1.5s	Sentence-level restructuring

Extended	1.5-2s	Higher-level planning begins
Prolonged	2-3s	Structural/argumentative planning
Very Long	> 3s	Deep planning or cognitive block

Table 1: Pause Duration Brackets

Pauses reflect cognitive processes in writing, with their duration and location indicating different planning levels. Short pauses at word boundaries are linked to word selection while longer pauses at sentence or paragraph boundaries are related to argument reorganisation. Prior studies found that stronger writers pause for shorter periods between words and more frequently between sentences, making pause distributions informative of writing quality [17]. Pause durations are thus derived by *compute_pause_distribution* which counts the number of inter-keystroke gaps during *Input* events and are grouped into eight brackets (Table 1).

Code	Boundary Type	Planning Level
<i>iw</i>	Within-word	Motor / lexical difficulty
<i>bww</i>	Between-word	Micro-planning, lexical access
<i>bws</i>	Between-sentence	Local structural planning
<i>bwp</i>	Between-paragraph	Global argumentative planning

Table 2: Linguistic Boundary Types

extract_pause_features then calculates summary statistics of pause behaviour and aggregates inter-keystroke gaps across linguistic boundary types (Table 2), where pauses are defined as the time between a key release and key press. These features enable the model to distinguish between lexical difficulty and structural planning.

5.5.3 Burst Analysis

Compositional fluency, the ability to produce sustained, uninterrupted text, is also a key predictor of writing quality. It can be measured by burst length, defined as the number of keystrokes before a longer pause. Previous works found that longer bursts indicate greater automaticity in lower-level processes such as typing and spelling, allowing writers to focus on higher-level tasks like planning and ideation [18]. Furthermore, writers differ in their revision behaviours, with the less skilled making frequent small local edits, and the more skilled making

fewer but larger and more deliberate revisions after longer stretches of text [19]. Thus, the structure of revision bursts acts as a key proficiency indicator for writing. To capture these patterns, *collect_bursts* identifies two types of writing sequences: productive bursts (Continuous *Input* or *Remove/Cut* **events with gaps $\leq 2000\text{ms}$**) and **rhythmic bursts (Continuous *Input*-only events)**. *extract_burst_features* then computes ten summary statistics for both burst length and duration of each burst type.

5.5.4 Cursor Revisit Features

Since writing is a recursive process characterised by planning, translation, transcription and evaluation, edits are not confined to local cursor positions. This explains why writers often trace back to insert transitions, elaborations or evidence, especially in argumentative essays where coherence is crucial. Moreover, higher-proficiency writers are more likely to engage in such non-linear compositions, while lower-proficiency writers often make corrections near their current cursor [20]. Despite this, cursor revisit is not used as a predictive feature in AES and thus, we introduced *cursor_revisit_features*, which counts how often each distinct cursor position is revisited and groups these counts into buckets (1, 2, 3, 4, 5, and ≥ 6 visits). This ensures that both the breadth and intensity of revision behaviour is captured, complementing deletion- and burst-based features in writing.

5.5.5 Activity & Event Counts

Function	Description	Features Produced
<i>get_counts_top1</i>	Counts occurrences of each writing activity type and specific key event for each writer	Activity types: <i>Input, Remove/Cut, Replace, Move, Paste, Nonproduction</i> (Section 5.2) Key events: <i>Space, Backspace, Enter, ArrowLeft, ArrowRight, Delete, comma, period, CapsLock</i> , and others
<i>build_keystroke_category_counts</i>	Replicates and extends <i>get_counts_top1</i> using Polars for efficient grouped counting	Counts across activity types, text change characters, and key events for each writer
<i>count_unique_key_stroke_categories</i>	Computes the number of distinct values per categorical column per writer	Behavioural diversity score, where a writer using a wider variety of key events and activity types likely demonstrates greater compositional sophistication

Table 3: Overview of Activity & Event Count Feature Functions

Three computational functions are implemented to count activity types, key events and behavioural diversity per writer, enabling the construction of behavioural fingerprints (Table 3). This is pivotal as the relative frequencies of activity types can uncover differences in writing behaviour across skill levels. For example, a high *Paste* count may indicate external text transfer, while a high *ArrowLeft* count relative to *Backspace* may suggest intentional revision using in-text navigation. A writer using a wider variety of key events and activity types is more likely to demonstrate greater compositional sophistication.

5.5.6 Session Normalised Ratios

Ratio	Formula	Interpretation
<i>DI_ratio</i>	$\frac{Input}{Input + Remove/Cut}$	Net Productivity Orientation: High values indicate focus on drafting, while lower values imply heavier revision focus
<i>text_key_ratio</i>	$\frac{Final\ Essay\ Length}{Total\ Keystrokes}$	Writing Efficiency: How much final text is produced per keystroke, penalises writers who type and delete extensively
<i>word_time_ratio</i>	$\frac{Final\ Word\ Count}{Session\ Time}$	Output Rate: Words produced per unit time, which is a direct measure of writing fluency
<i>word_event_ratio</i>	$\frac{Final\ Word\ Count}{Total\ Events}$	Compositional Efficiency: How much final text results from each discrete writing event
<i>event_time_ratio</i>	$\frac{Total\ Events}{Session\ Time}$	Writing Activity Density: Captures how actively a writer engages with the editor across the session

Table 4: Session-Normalised Ratio Features and Their Interpretations

Using raw event counts can be misleading as a writer who types more will naturally have more edits and pauses even if their writing behaviours are similar to one who types less. Hence, for fair comparison across writers with different session lengths, these counts are normalised by total keystrokes, session time or total events (Table 4), a standard approach in AES research [21]. In particular, *DI_ratio* and *word_time_ratio* are commonly used AES metrics. Prior work found that writers who scored higher had lower proportions of deletions and higher proportions of insertions [16], suggesting a more additive writing process, while also producing more words per minute than their lower-scoring counterparts [21]. These findings thus motivate our inclusion of both ratios as our feature set.

5.5.7 Editing Run Statistics

Dimension	Statistics Computed	Interpretation
Run Length	Mean, Median, Std, Min, Max	Distinguishes writers who produce text in long, steady streams from those who write in short, fragmented bursts
Run Duration	Mean, Median, Std, Min, Max	Captures time spent in each drafting or revision phase, reflecting sustained engagement vs frequent interruptions
Inter-Run Gap	Mean, Median, Std, Min, Max	Measures the cost of mode-switching between drafting and revision, as longer gaps may indicate greater cognitive load at transition points

Table 5: Dimensions and Interpretations of Editing Run Statistics

As discussed in Section 5.5.4, writing involves alternating between drafting and revision. Skilled writers are found to produce longer, more sustained stretches of text with periodic revisions, while less proficient writers show more fragmented writing with frequent corrections [22]. *compute_editing_run_stats* is therefore implemented to preserve these temporal traits. It identifies the consecutive runs of the same event type, including *Input* (continuous drafting) and *Remove/Cut* (continuous revision), before computing descriptive statistics across three dimensions (Table 5). Efficient writers are expected to have longer *Input* runs, short *Remove/Cut* runs and smaller inter-run gaps due to progressive and fluent composition, while weaker writers are more likely to engage in effortful and heavily monitored writing characterised by short, irregular runs and wide gaps [23].

6. Methodology

Our solution is a two-stage stacking ensemble. Stage 1 trains five independent base models, with each generating OOF predictions via 10-fold CV, and a sixth OOF contributed by the Ridge regression text signal (Section 5.4). Stage 2 combines these six features using two complementary meta-learners whose outputs are blended via inverse-RMSE weighting.

6.1 Stage 1: Base Regression Models

Three gradient-boosted tree regressors, LightGBM, XGBoost and Catboost, form our base layer, chosen due to their ability to handle non-linear relationships. All hyperparameters were tuned using Optuna by minimising mean OOF RMSE across all 10 folds instead of a single validation split, a methodological distinction to prevent hyperparameters from overfitting any single fold, producing a more reliable estimate of generalisation performance [24].

6.1.1 LightGBM (CV RMSE: 0.5926)

LightGBM is computationally efficient and handles large datasets effectively. It uses leaf-wise tree growth, which finds the globally best split at each step, allowing deeper partitions than level-wise methods and improving prediction accuracy [25]. We deliberately set our tuned configuration at *max_depth*=3, *num_leaves*=314 as the shallow *max_depth* limits each path to three splits, reducing overfitting on this relatively small dataset. Due to LightGBM's leaf-wise growth, many leaves can still exist at shallow depth, which leads to complex feature partitions. Hence, the high *num_leaves* helps to maintain model capacity. Together, these parameters balance regularisation and model expressiveness. Since each fold varies in difficulty, early stopping of 200 rounds with a maximum of 10 000 boosting rounds was applied to adaptively determine the number of trees, allowing simpler folds to stop earlier and more complex ones to train longer. This reduces overfitting to fold-specific noise, allowing the model to achieve a CV RMSE of 0.5926.

6.1.2 XGBoost (CV RMSE: 0.8624)

XGBoost is an L1/L2-regularised gradient boosting model that employs level-wise tree growth, combining multiple weak learners for improved generalisation [26]. Its tuned configuration uses a very low learning rate of 0.00342 with *max_depth*=3, slowing learning to mitigate overfitting on a small dataset. This resulted in the weakest performance among the three base learners, with a CV RMSE of 0.8624. Despite this, XGBoost was retained in the ensemble for its diversity. Its level-wise growth strategy is different from LightGBM's leaf-wise approach, meaning its errors are likely uncorrelated with those of the stronger base learners, a property that stacking explicitly exploits, as ensemble diversity matters more than individual model strength. Since stacking tolerates weak base learners [27] provided their errors are sufficiently independent, the performance gap is of less concern. The meta-learner receives all base OOF predictions as inputs and can automatically learn to down-weight [28] XGBoost's contribution if its signal proves redundant, as reflected in its fitted coefficients.

6.1.3 CatBoost (CV RMSE: 0.5872)

CatBoost uses symmetric (oblivious) trees with ordered boosting, reducing target leakage during gradient estimation and producing well-calibrated predictions [29]. Its tuned configuration uses a Bayesian bootstrap with *bagging_temperature*=0.998 for near-maximal stochasticity in sample weighting [30]. This increases prediction diversity relative to LightGBM and XGBoost, whose sub-sampling is more fixed, by varying its base prediction across folds, reducing correlation in the ensemble and improving ensemble generalisation. Thus, it achieved the strongest individual performance with a CV RMSE of 0.5872.

6.2 Ordinal Classification Models (LightGBM RMSE: 0.6011 | XGBoost RMSE: 0.5935)

To capture the ordered nature of essay scores (Section 3.2), two additional base models, LightGBM and XGBoost, are trained as ordinal classifiers. The target has 11 discrete levels (0.5 to 6.0 in 0.5 increments) and is reformulated into a series of binary tasks, whereby for each threshold $c \in \{0.5, 1.0, \dots, 5.5\}$, the classifier predicts whether the essay score exceeds c . These thresholds correspond to boundaries between adjacent score levels, allowing each model to

learn one level of separation at a time. A final continuous prediction is obtained by summing the exceedance probabilities:

$$\hat{y} = 0.5 + 0.5 \sum_{c=0.5}^{5.5} P(\text{score} > c)$$

This follows the proportional-odds model, ensuring predictions respect the ordinal structure of the target, giving an OOF RMSE of 0.6011 for LightGBM and 0.5935 for XGBoost.

6.3 Ridge Regression Text Signal

A Ridge regression model is also trained on the full, uncompressed word n-gram TF-IDF matrix (Section 5.4), contributing a sixth OOF column to the meta-layer. Unlike the SVD-compressed 64-dimensional embeddings used by tree models, this signal is derived directly from the full high-dimensional sparse lexical space. This is intentional, as tree models require SVD compression to handle sparsity effectively, but inevitably discards information outside the top components [31]. Thus, the Ridge model recovers this lost lexical signal by operating on the full feature space, providing the meta-learner with complementary lexical information at a different granularity. This design is rare as most solutions use either SVD-based tree models or full-space linear models, but not both.

6.4 Cross-Validation Strategy

All five base models and the Ridge signal were trained using 10-fold stratified CV (*StratifiedKFold, random state 123*). With 2471 essay samples, a 10-fold split allocates approximately 2224 training and 247 validation samples per fold. This is preferred over smaller folds [32], such as 5-fold, which would yield smaller validation sets of about 494 samples, producing noisier per-fold RMSE estimates, higher OOF prediction variance and less reliable early-stopping signals. 10-fold thus offers a better bias-variance tradeoff for fold-level performance estimation given our sample size. To tackle the issue of missing ordinal classification in regression (Section 3.2), folds were stratified using the variable, *strata = (score × 2).astype(int)*, mapping each score increment of 0.5 to a unique integer bin, yielding 11 ordinal strata. This ensures representation of all score levels in each fold [33], including the underrepresented tails (1.0-2.0 and 5.0-6.0), preventing folds from excluding extreme scores. The resulting OOF predictions from all five models and the Ridge signal form a meta-feature matrix of shape (2471 × 6) that is the input to Stage 2.

6.5 Stage 2: Meta-Learning

The 6-column OOF matrix is passed to two complementary meta-learners. A third boosted tree was not used, as it would duplicate the inductive bias of Stage 1 models. Instead, linear regression and ordinal logistic regression provide complementary perspectives, one operating in prediction space and the other in log-odds space.

6.5.1 Linear Regression (OOF RMSE: 0.5792)

Linear Regression is the primary meta-learner. With 6 input columns and 2471 samples, its risk of overfitting is low, making regularisation redundant. Its predictions are also already well-

calibrated, real-valued score estimates, making linear combination a natural aggregation method [34]. The learned coefficients also reveal the base models’ contribution: XGBoost received the lowest weight, consistent with its higher RMSE, without manual intervention.

6.5.2 Ordinal Logistic Regression Meta-Learner (OOF RMSE:0.5802)

Ordinal Logistic Regression is the secondary meta-learner, applying the same proportional-odds framework at the meta level. Unlike the base ordinal classifiers, which operate on more than 300 raw features, this model uses only the 6 OOF predictions, learning from the combined outputs of all base models. Its goal is to answer a higher-level question: “Given all model evidence, which score thresholds are most likely exceeded?” A key advantage of operating in log-odds space is that it can capture calibration differences [35] between meta-features that a linear model may miss. For example, if LightGBM and CatBoost both predict 4.0, but the ordinal classifiers show high probabilities of exceeding 3.5 and 4.0 and low probabilities of exceeding 4.5, the ordinal meta-learner can infer a score just below 4.5. This captures finer threshold-level structure that a linear model may miss. The same 11 thresholds (0.5 to 5.5 in 0.5 increments) are used, matching the score boundaries.

6.6 Model Blending and Final Prediction (Ensemble RMSE: 0.57903)

The two meta-learner OOF predictions are blended using inverse-RMSE weighting. This assigns higher weight to the better-performing meta-learner with lower OOF error, without manual tuning, resulting in the final computed weights of 0.5003 (linear) and 0.4997 (logistic). There is a minimal difference of 0.0008 between the two weights, suggesting near-equal performance and reinforcing the idea that both models extract similar signals from the same OOF features.

	Equation	Linear Meta-Model OOF	Logistic Meta-Model OOF
Weight	$w_i = \frac{1/RMSE_i}{\sum(1/RMSE)}$	0.5004	0.4996

Table 6. Linear regression and ordinal logistic meta-model resulting weights

The inverse-RMSE scheme also provides robustness: if one meta-learner performs worse, it is automatically down-weighted without any redesign [36]. Final predictions are clipped to the valid score range [0.5, 6.0] to prevent out-of-range outputs without introducing additional model complexity. The final ensemble achieves a CV RMSE of 0.57903, outperforming all individual base models.

6.7.1 Other Models Considered but not Utilised

Several other ensemble and deep learning algorithms were explored during development but were ultimately not adopted. First, LightAutoML, an ensemble incorporating algorithms such as linear models, gradient boosting and neural networks, was explored given its strong reported performance in this Kaggle competition. However, we experienced difficulties with the installation and linking the LightAutoML model to the competition, which led to us abandoning

the approach. Beyond LightAutoML, the use of a Multi Layer Perceptron (MLP) neural network was also explored. A cross-validation loop with feature engineering through the use of a SimpleImputer and RobustScaler¹ was implemented. However, the initial run of the MLP neural network yielded performance that fell below the 70th percentile benchmark, suggesting that the tabular, sparse-feature nature of the writing process data did not provide sufficient signal for deep learning to be effective [37]. Given these constraints, the final pipeline was streamlined around gradient boosting and linear models, which offered more stable and interpretable performance on this dataset.

7. Experimental Results

7.1 Summary of Results

The table below summarises the performance of all models, benchmarked against the naive mean-prediction baseline that is equal to the score standard deviation (RMSE $\approx$ 1.024).

Model	Type	CV RMSE	Fold RMSE Range	OOB RMSE	Δ vs. Baseline
LGBM Regressor	Base	0.59232	0.53460 – 0.63823		−0.432
XGBoost Regressor	Base	0.86239	0.83938 – 0.87315		−0.162
CatBoost Regressor	Base	0.58721	0.53265 – 0.62545		−0.437
LGBM Classifier	Base			0.59850	−0.426
XGBoost Classifier	Base			0.59347	−0.431
Ridge (TF-IDF)	Signal				
Linear Meta-Model	Stage 2			0.57964	−0.445

¹ SimpleImputer: re-handles any remaining NaNs by replacing them with the column mean
RobustScaler: scales features using median/interquartile range, making it resistant to outliers

Ordinal Logistic Meta-Model	Stage 2			0.58022	−0.444
Final Ensemble	Blend	0.57903			−0.445

Table 7. Performance summary across all pipeline stages. Baseline RMSE ≈ 1.024 (naïve mean prediction)

Among the regressors, CatBoost had the lowest CV RMSE (0.58721) and the most stable fold range (0.53265–0.62545), showing strong generalisation. LightGBM performed similarly, with a CV RMSE (0.59232) and a consistent fold range. In contrast, XGBoost performed the worst with a very high CV RMSE (0.86239) despite its narrow fold range, suggesting a weaker fit due to sensitivity to the sparse high-dimensional TF-IDF features. For ordinal classifiers, both LightGBM (OOF RMSE: 0.59850) and XGBoost (OOF RMSE: 0.59347) achieved competitive performance, confirming that reformulating regression into ordinal binary decisions can produce useful complementary signals. At the meta level, both linear (0.57964) and ordinal logistic (0.58022) regressors outperformed all base models, with the final inverse-RMSE blend (weights: linear 0.5003, logistic 0.4997) achieving the best overall CV RMSE of 0.57903. This confirms the idea that base models can capture complementary patterns effectively exploited through stacking.

7.2 Ablation Analysis

We conducted an ablation study to assess the contribution of each feature group by systematically removing them and recording the change in RMSE, using the full-ensemble baseline RMSE of 0.57903 (Table 8). While improvements in RMSE may appear small, they are meaningful in our context: the RMSE gap between the top 2% and 5% submissions is less than 0.005. Furthermore, each +0.001 RMSE corresponds to roughly one additional essay mispredicted by more than 0.5 points, making every small decimal gain consequential.

Feature Group Removed	RMSE	Change in RMSE	Estimated Essays Mispredicted
Text-Based Features	0.58297	+0.00394	~10 additional
Keystroke Process Features	0.58537	+0.00634	~16 additional
Pause distribution & Burst-Related Features	0.57922	+0.00019	~1 additional

Table 8. RMSE changes with feature group ablation

The removal of text-based features (+0.00394) produced the second-largest degradation, indicating that textual information plays an important role in predicting essay quality. This is because the TF-IDF features capture vocabulary-level writing quality that surface and grammar features cannot fully encapsulate on their own. Secondly, removing all keystroke process

features (+0.00634) led to the greatest degradation. This suggests that behavioural signals such as typing speed, pause patterns and revision behaviour provide complementary information about *how* the essay was written, an aspect unrecoverable by final texts. Lastly, the removal of pause distribution and burst-related features (+0.00019) contributed the smallest but still meaningful degradation, suggesting that temporal writing patterns are more informative than simple keystroke counts. The relatively smaller increase is likely attributed to partial overlap with broader keystroke features instead of redundancy.

These results validate our dual-modality design, demonstrating the complementary nature of text- and process-based features, yielding the strongest performance when combined.

7.3 Kaggle Ranking & Prediction Score

Submission and Description			Private Score	Public Score	
notebookb28e7f61ca - Version 3 Succeeded (after deadline) · 10h ago · Notebook notebookb28e7f61ca Version 3			0.562428	0.576670	
35	Martin Bielik		0.576557	191	2y
36	TeamShiny in Akihabara		0.576570	61	2y
37	Emeraid		0.576608	28	2y
38	Non-Linear		0.576686	175	2y
39	rotto		0.576689	84	2y

Figure 6: Kaggle scoring ranking

Our final submission achieved a public score of 0.576670, which corresponds to Rank 38 out of 1877 entries, placing the team in the Top 2.02% on the public leaderboard and in the top 10% band (Figure 6). This result further validates the effectiveness of our stacking architecture, stratified CV scheme and dual-modality feature design in producing a well-generalised solution that performs competitively with other top-tier submissions.

8. Solution Novelty

8.1 Dual-Modality Design

Most AES systems rely on the final essay text for holistic score prediction. However, this data is not available for us, prompting us to adopt a novel, dual-modality feature engineering approach that analyses writing from two complementary perspectives: (i) what was written (using textual and content signals) and (ii) how it was written (using behavioural and process signals). The first modality directly addressed the problem of missing final texts by reconstructing essays. It combines surface features, punctuation error detection, and TF-IDF with Ridge regression to capture the linguistic and structural properties of the written output, ensuring that content quality and organisation can still be modelled despite incomplete data access. The second modality addressed the problem of linking writing behaviour to quality,

where features like pause distributions revealed planning depth, cognitive load, and real-time decision-making, providing process-level signals that are absent from text alone. These two modalities are thus complementary and address limitations in the data. Text-based features are sensitive to essay topic and prompt variation, while behavioural features are largely topic-agnostic, capturing writing patterns independent of subject matter. Together, they cover different dimensions of the dataset, improving both feature expressiveness and generalisation across diverse prompts and writing contexts.

8.2 Ordinal Classification Stacking

Since conventional regression models ignore the ordinal structure of essay scores and treat them as continuous, we reformulated the task as a series of binary classification problems. Logistic regression models are trained to estimate the probability of an essay score exceeding the set threshold, allowing the ensemble to exploit the ordinal structure at both the base level (ordinal classifiers in Stage 1) and the meta level (ordinal logistic meta-learner in Stage 2). This captures both the continuous magnitude and the discrete ordering of scores, improving the model's ability to distinguish between closely-scored essays.

8.3 Inverse-RMSE Weighted Meta-Blending

Unlike conventional ensemble methods that use simple averaging or manually tuned weights, our approach utilises a blended meta-learner that automatically assigns weights based on each model's OOF RMSE. This makes the framework model-agnostic, supporting both linear regression and logistic regression meta-learners. Its self-adjusting nature is also advantageous because if model parameters change, the weights will update automatically to reflect the new relative performance, improving generalisation on unseen data [38]. This approach has been validated in ensemble forecasting literature as a robust alternative to uniform averaging.

9. Conclusion and Reflection

This project exposed us to the practical challenges of real-world machine learning, such as feature engineering from noisy, anonymised data and effective hyperparameter tuning. As this was our first Kaggle competition, we also encountered issues like metric exploitation, prompting us to pivot to another topic mid-project. Through extensive research, experimentation and careful OOF RMSE validation, we were also able to successfully demonstrate that writing quality can be predicted using keystroke behaviour alone. This can be validated by our top 2% ranking (38/1,877) on the Kaggle leaderboard for our two-stage stacking ensemble, highlighting the effectiveness of our dual-modality design and the complementary nature of text and behavioural features. We also gained technical insights, such as the importance of modelling ordinal structure, using OOF-based tuning to reduce overfitting and leveraging stacking with diverse error patterns. Overall, the project highlights the value of experimentation with feature design in building robust predictive systems.

10. References

[1] GeeksforGeeks, "Overfitting in Decision Tree Models," GeeksforGeeks. Accessed: Mar. 31, 2026 [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/overfitting-in-decision-tree-models/>

- [2] “notebook1f66b953aa,” Kaggle.com. Accessed: Mar. 12, 2026. [Online.] Available: <https://www.kaggle.com/code/tohjunsheng/notebook1f66b953aa>
- [3] yunsuxiaozi, “Writing Quality Top 1 study notebook,” [Kaggle.com](https://www.kaggle.com/code/yunsuxiaozi/writing-quality-top-1-study-notebook/notebook). Accessed: Mar. 16, 2026. [Online.] Available: . <https://www.kaggle.com/code/yunsuxiaozi/writing-quality-top-1-study-notebook/notebook>
- [4] C. T. Lim, C. H. Bong, W. S. Wong, and N. K. Lee, “A Comprehensive Review of Automated Essay Scoring (AES) Research and Development,” *Pertanika Journal of Science and Technology*, vol. 29, no. 3, Jul. 2021, doi: <https://doi.org/10.47836/pjst.29.3.27>.
- [5] M. Sayed, S. Eltanbouly, M. Bashendy, and T. Elsayed, “Feature Engineering is not Dead: A Step Towards State of the Art for Arabic Automated Essay Scoring,” *Proceedings of The Third Arabic Natural Language Processing Conference*, pp. 231–245, 2025, doi: <https://doi.org/10.18653/v1/2025.arabicnlp-main.19>.
- [6] K. Barkaoui and J. Woodworth, “An exploratory study of the construct measured by automated writing scores across task types and test occasions,” *Studies in Language Assessment*, vol. 12, no. 1, pp. 26–63, 2023, doi: <https://doi.org/10.58379/qcfs2805>.
- [7] W. Li and H. Liu, “Applying large language models for automated essay scoring for non-native Japanese,” *Humanities & social sciences communications*, vol. 11, no. 1, Jun. 2024, doi: <https://doi.org/10.1057/s41599-024-03209-9>.
- [8] J. Fleckenstein, J. Meyer, T. Jansen, S. Keller, and O. Köller, “Is a Long Essay Always a Good Essay? The Effect of Text Length on Writing Assessment,” *Frontiers in Psychology*, vol. 11, Sep. 2020, doi: <https://doi.org/10.3389/fpsyg.2020.562462>.
- [9] F. Shadloo, H. S. Ahmadi, and B. Ghonsooly, “Exploring syntactic complexity and its relationship with writing quality in EFL argumentative essays,” *Topics in Linguistics*, vol. 20, no. 1, pp. 68–81, Jun. 2019, doi: <https://doi.org/10.2478/topling-2019-0005>.
- [10] Z. Ke and V. Ng, “Automated Essay Scoring: A Survey of the State of the Art,” *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*, Aug. 2019, doi: <https://doi.org/10.24963/ijcai.2019/879>.
- [11] K. Doi, K. Sudoh, S. Nakamura, and T. Watanabe, “Enhancing Automated Essay Scoring with Grammatical Features using Multi-task Learning and Item Response Theory,” *Journal of Natural Language Processing*, vol. 32, no. 2, pp. 438–479, 2025, doi: <https://doi.org/10.5715/jnlp.32.438>.
- [12] A. Doewes, “Rethinking Automated Essay Scoring: Agreement, Fairness, and Feedback,” Eindhoven University of Technology, 2026.
- [13] Ammar Ismael Kadhimi, Y.-N. Cheah, Inaam Abbas Hieder, and Rawaa Ahmed Ali, “Improving TF-IDF with Singular Value Decomposition (SVD) for Feature Extraction on Twitter,” *International Engineering Conference*, Jan. 2017, doi: <https://doi.org/10.23918/iec2017.16>.
- [14] A. Kulmizev et al., “The Power of Character N-grams in Native Language Identification,” *ACLWeb*, Sep. 01, 2017. <https://aclanthology.org/W17-5043/>
- [15] K. Barkaoui, “WHAT CAN L2 WRITERS’ PAUSING BEHAVIOR TELL US ABOUT THEIR L2 WRITING PROCESSES?,” *Studies in Second Language Acquisition*, vol. 41, no. 3, pp. 529–554, 2019. doi:10.1017/S027226311900010X

- [16] Y. Tian, S. Crossley, and L. Van Waes, "The KLiCke corpus: Keystroke logging in compositions for knowledge evaluation," *Journal of Writing Research*, vol. 17, no. 1, pp. 23–60, Jun. 2025, doi: <https://doi.org/10.17239/jowr-2025.17.01.02>.
- [17] F. Zarrabi, M. H. Fadavi Amiri, and H. Bozorgian, "Effects of the longest pause, its location, and pause variance on successful EFL writing performance across writing tasks with diverse degrees of complexity," *System*, vol. 110, p. 102929, Sep. 2022, doi: <https://doi.org/10.1016/j.system.2022.102929>.
- [18] S. Hurschler Lichtsteiner, W. Wicki, and P. Falmann, "Impact of handwriting training on fluency, spelling and text quality among third graders," *Reading and Writing*, vol. 31, no. 6, pp. 1295–1318, Jan. 2018, doi: <https://doi.org/10.1007/s11145-018-9825-x>.
- [19] R. Conijn, C. Cook, M. van Zaanen, and L. Van Waes, "Early prediction of writing quality using keystroke logging," *International Journal of Artificial Intelligence in Education*, vol. 32, Aug. 2021, doi: <https://doi.org/10.1007/s40593-021-00268-w>.
- [20] Fabio Cerpelloni, "What Skilled and Unskilled Writers Do (According to Research)," Medium. Accessed: Apr. 12, 2026. [Online.] Available: <https://medium.com/technical-excellence/what-skilled-and-unskilled-writers-do-according-to-research-3292606d88f3>
- [21] M. Zhang, M. Zhu, P. Deane, and H. Guo, "Identifying and Comparing Writing Process Patterns Using Keystroke Logs," *Springer Proceedings in Mathematics & Statistics*, pp. 367–381, 2019, doi: https://doi.org/10.1007/978-3-030-01310-3_32.
- [22] N. Vandermeulen, M. Leijten, and L. Van Waes, "Reporting Writing Process Feedback in the Classroom. Using Keystroke Logging Data to Reflect on Writing Processes," *Journal of Writing Research*, vol. 12, no. vol. 12 issue 1, pp. 109–140, Jun. 2020, doi: <https://doi.org/10.17239/jowr-2020.12.01.05>.
- [23] I. Ivaska, O. Toropainen, and S. Lahtinen, "Pauses during a writing process in two typologically different languages," *Journal of Writing Research*, vol. 16, no. 3, pp. 407–433, Feb. 2025, doi: <https://doi.org/10.17239/jowr-2025.16.03.03>.
- [24] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," *arXiv (Cornell University)*, Jul. 2019, doi: <https://doi.org/10.48550/arxiv.1907.10902>.
- [25] A. Singh, "LightGBM Explained: Faster and Smarter Gradient Boosting," Medium. Accessed: Apr. 12, 2026. [Online.] Available: <https://medium.com/@abhaysingh71711/lightgbm-explained-faster-and-smarter-gradient-boosting-4ab549629925>
- [26] XGBoosting, "XGBoost Advantages and Disadvantages (pros vs cons) | XGBoosting," [Xgboosting.com](https://xgboosting.com). Accessed: Apr. 12, 2026. [Online.]. Available: <https://xgboosting.com/xgboost-advantages-and-disadvantages-pros-vs-cons/>
- [27] M. Liang et al., "A Stacking Ensemble Learning Framework for Genomic Prediction," *Frontiers in Genetics*, vol. 12, Mar. 2021, doi: <https://doi.org/10.3389/fgene.2021.600040>.
- [28] S. Farhadi, Samuele Tatullo, and F. Ferrian, "Comparative analysis of ensemble learning techniques for enhanced fatigue life prediction," *Scientific Reports*, vol. 15, no. 1, Apr. 2025, doi: <https://doi.org/10.1038/s41598-024-79476-y>.
- [29] A. Singh, "CatBoost: A High-Performance Gradient Boosting for Categorical Data," Medium. Accessed: Apr. 12, 2026. [Online.] Available:

<https://medium.com/@abhaysingh71711/catboost-a-high-performance-gradient-boosting-for-categorical-data-875bf6060cb6>

[30] G. Gibson, “Bayesian Conformal Prediction via the Bayesian Bootstrap,” arXiv.org, 2025. <https://arxiv.org/abs/2508.01418>.

[31] H. D. Thi, K. H. Manh, V. T. Anh, T. P. T. Quynh, and T. N. Viet, “Dimensionality Reduction with Truncated Singular Value Decomposition and K-Nearest Neighbors Regression for Indoor Localization,” *International Journal of Advanced Computer Science and Applications*, vol. 14, no. 10, 2023, doi: <https://doi.org/10.14569/ijacsa.2023.0141034>.

[32] T. Abedin, H. Xu, and S. Uddin, “The impact of K selection in K-fold cross-validation on bias and variance in supervised learning models,” *Scientific Reports*, p. -, Jan. 2026, doi: <https://doi.org/10.1038/s41598-026-37247-x>.

[33] K. Fujiwara, “Knowledge distillation with resampling for imbalanced data classification: Enhancing predictive performance and explainability stability,” *Results in Engineering*, vol. 24, p. 103406, Nov. 2024, doi: <https://doi.org/10.1016/j.rineng.2024.103406>.

[34] P. McCullagh, “Regression Models for Ordinal Data,” *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 42, no. 2, pp. 109–127, Jan. 1980, doi: <https://doi.org/10.1111/j.2517-6161.1980.tb01109.x>.

[35] M. Edlinger, Maarten van Smeden, H. F. Alber, M. Wanitschek, and Ben Van Calster, “Risk prediction models for discrete ordinal outcomes: Calibration and the impact of the proportional odds assumption,” *Statistics in Medicine*, vol. 41, no. 8, pp. 1334–1360, Dec. 2021, doi: <https://doi.org/10.1002/sim.9281>.

[36] M. Noor and Meherab, Md Muntaqim, “Regularized Meta-Learning for Improved Generalization,” arXiv.org, 2026. <https://arxiv.org/abs/2602.12469>

[37] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why do tree-based models still outperform deep learning on tabular data?,” arXiv.org, Jul. 2022. <https://arxiv.org/abs/2207.08815>

[38] X. Wang, R. J. Hyndman, F. Li, and Y. Kang, “Forecast combinations: An over 50-year review,” *International Journal of Forecasting*, vol. 39, no. 4, Dec. 2022, doi: <https://doi.org/10.1016/j.ijforecast.2022.11.005>.